Java Developer Interview

Question Sheet

5 Years of Experience | Target · Rakuten · Wells Fargo

Topics: Core Java · Spring Boot · Microservices · DSA · SQL · Design Patterns · LLD

Difficulty: Easy | Med | Hard

---

1. Core Java

OOP & Language Fundamentals

1. [Easy] What are the four pillars of OOP? Explain with examples.
2. [Easy] Difference between abstract class and interface. When to use which?
3. [Easy] What is method overloading vs method overriding?
4. [Med] Explain SOLID principles with one real-world example each.
5. [Med] What is the diamond problem? How does Java 8 resolve it with default methods?
6. [Med] Difference between composition and inheritance. Which is preferred and why?
7. [Easy] What is the significance of the final keyword on class, method, and variable?
8. [Easy] Explain static keyword — static methods, static variables, static blocks.
9. [Easy] What is covariant return type? Give an example.
10. [Med] Explain the use of the instanceof operator and pattern matching (Java 16+).

Collections Framework

11. [Med] Explain the internal working of HashMap (hashing, buckets, load factor, rehashing).
12. [Med] What happens when two keys have the same hashCode in a HashMap?
13. [Med] Difference between HashMap, LinkedHashMap, TreeMap, and ConcurrentHashMap.
14. [Med] Internal implementation of ConcurrentHashMap — how does it achieve thread safety?
15. [Easy] Difference between HashSet and TreeSet. How does HashSet internally work?
16. [Easy] ArrayList vs LinkedList — time complexity for add, remove, get operations.
17. [Med] What is fail-fast vs fail-safe iterator? Give examples.
18. [Med] Explain Comparable vs Comparator. How would you sort a list of custom objects?
19. [Hard] How does TreeMap maintain sorted order? What is the underlying data structure?
20. [Med] What is the difference between poll() and remove() in a Queue?

Generics, Streams & Lambdas (Java 8+)

21. [Med] What are generics? Explain bounded wildcards: <? extends T> vs <? super T>.
22. [Med] Explain the Stream API — difference between intermediate and terminal operations.
23. [Med] What is the difference between map() and flatMap() in Stream API?
24. [Easy] How does Optional work? When should you use it?
25. [Med] Explain method references — types with examples.
26. [Hard] What is a functional interface? Write a custom functional interface with a lambda.
27. [Med] Difference between Stream.collect(Collectors.toList()) and .toUnmodifiableList().
28. [Easy] What are default and static methods in interfaces (Java 8)?
29. [Med] Explain Stream.parallel() — when is it beneficial vs harmful?
30. [Hard] What is the difference between Predicate, Function, Consumer, and Supplier?

Multithreading & Concurrency

31. [Med] Explain the lifecycle of a thread in Java.
32. [Med] Difference between synchronized method and synchronized block. Which is better?

33. **[Hard]** What is a deadlock? How do you detect and prevent it?
34. **[Hard]** Explain the volatile keyword. What problem does it solve?
35. **[Hard]** Difference between wait(), notify(), notifyAll(). Explain with a producer-consumer example.
36. **[Med]** What is the Executor framework? Types of thread pools — when to use each?
37. **[Hard]** What is the happens-before relationship in the Java Memory Model?
38. **[Hard]** Explain ReentrantLock vs synchronized. What extra features does ReentrantLock provide?
39. **[Med]** What is CountDownLatch vs CyclicBarrier? Practical use cases?
40. **[Hard]** How does AtomicInteger work internally (CAS — Compare and Swap)?
41. **[Med]** What is a Future vs CompletableFuture? Explain chaining with thenApply().
42. **[Hard]** What is ThreadLocal? When would you use it?

Memory Management & JVM

43. **[Med]** Explain JVM architecture — ClassLoader, JIT, Heap, Stack, Metaspace.
44. **[Med]** What are the different regions of Java Heap memory (Eden, Survivor, Old Gen)?
45. **[Med]** Explain Garbage Collection — Serial, Parallel, G1, ZGC collectors.
46. **[Hard]** What is a memory leak in Java? How would you detect one?
47. **[Easy]** Difference between stack memory and heap memory.
48. **[Med]** What does the finalize() method do? Why is it deprecated?
49. **[Hard]** How would you tune JVM for a high-throughput application? Which GC flags?
50. **[Med]** What is a strong, soft, weak, and phantom reference?

2. Spring Boot & Spring Framework

Core Spring Concepts

51. **[Easy]** What is Dependency Injection? Types of DI in Spring.
52. **[Easy]** Difference between @Component, @Service, @Repository, @Controller.
53. **[Med]** Explain Spring Bean lifecycle — instantiation to destruction.
54. **[Med]** What are the scopes of a Spring bean? Which is default?
55. **[Med]** Explain @Autowired — how does Spring resolve ambiguity? What is @Qualifier?
56. **[Med]** What is the difference between @Bean and @Component?
57. **[Med]** What is @Configuration and how does it differ from @Component?
58. **[Hard]** Explain Spring AOP — concepts: aspect, advice, joinpoint, pointcut, weaving.
59. **[Med]** What is @Transactional? Explain propagation levels — REQUIRED, REQUIRES_NEW, NESTED.
60. **[Hard]** What is circular dependency in Spring? How can you fix it?

Spring Boot Specifics

61. **[Easy]** What is auto-configuration in Spring Boot? How does @SpringBootApplication work?
62. **[Med]** What is Spring Boot Actuator? Key endpoints and their uses.
63. **[Med]** Explain Spring Boot externalized configuration — order of property resolution.
64. **[Easy]** Difference between application.properties and application.yml.
65. **[Med]** How do you create custom auto-configuration in Spring Boot?
66. **[Med]** What is a Spring Boot Starter? How to create a custom starter?
67. **[Easy]** Explain @ConditionalOnProperty, @ConditionalOnClass annotations.
68. **[Med]** How does Spring Boot handle exception globally? @ControllerAdvice, @ExceptionHandler.
69. **[Med]** What is Spring Boot DevTools? How does live reload work?
70. **[Hard]** Explain how embedded Tomcat works in Spring Boot. How would you replace it with Jetty?

Spring Data & Persistence

71. **[Med]** Explain Spring Data JPA — difference between JpaRepository, CrudRepository, PagingAndSortingRepository.
72. **[Med]** What is Hibernate first-level and second-level cache?
73. **[Med]** N+1 problem in Hibernate — what is it and how to fix it?
74. **[Hard]** Explain FetchType.LAZY vs FetchType.EAGER. When does LazyInitializationException occur?
75. **[Med]** How does @Transactional work with Hibernate sessions?
76. **[Med]** Difference between JPQL, Criteria API, and native SQL in Spring Data.
77. **[Hard]** What is optimistic vs pessimistic locking? How to implement in JPA?
78. **[Med]** What is database connection pooling (HikariCP)? Key configuration params.

Spring Security

79. **[Med]** How does Spring Security filter chain work?
80. **[Med]** Explain JWT-based authentication — token structure, validation flow.
81. **[Med]** What is OAuth2 in Spring Security? Difference between authorization code and client credentials flow.
82. **[Hard]** How do you implement role-based access control (RBAC) in Spring?
83. **[Med]** Explain CSRF protection — what is it and when would you disable it (e.g., stateless REST APIs)?

3. Microservices & System Design

Microservices Patterns & Design

84. **[Med]** What are the key principles of microservices architecture?
85. **[Med]** Explain the Saga pattern — choreography vs orchestration.
86. **[Hard]** What is the CQRS pattern? When would you apply it?
87. **[Hard]** Explain Event Sourcing. How does it differ from traditional CRUD?
88. **[Med]** What is the API Gateway pattern? How does it differ from a load balancer?
89. **[Med]** What is service discovery? Difference between client-side (Eureka) and server-side discovery.
90. **[Hard]** How do you handle distributed transactions in microservices?
91. **[Med]** Explain the Circuit Breaker pattern — Closed, Open, Half-Open states. (Resilience4j)
92. **[Med]** What is the Strangler Fig pattern? When do you use it?
93. **[Hard]** How would you decompose a monolith into microservices? What strategy?

REST APIs & Communication

94. **[Easy]** What are REST constraints? Difference between REST and SOAP.
95. **[Med]** HTTP methods — idempotency of GET, PUT, DELETE, POST, PATCH.
96. **[Med]** What is HATEOAS? Practical usage in Spring.
97. **[Med]** Difference between synchronous (REST/gRPC) and asynchronous (Kafka/RabbitMQ) communication.
98. **[Med]** How do you version a REST API? Strategies with trade-offs.
99. **[Easy]** HTTP status codes — 200, 201, 204, 400, 401, 403, 404, 409, 422, 500, 503.
100. **[Med]** What is gRPC? When would you choose it over REST?
101. **[Med]** How would you design a rate limiter for an API?

Messaging — Kafka & RabbitMQ

102. **[Med]** Explain Kafka architecture — brokers, topics, partitions, consumer groups, offsets.
103. **[Hard]** How does Kafka guarantee ordering? At-least-once vs exactly-once delivery.
104. **[Med]** Difference between Kafka and RabbitMQ. When to choose each?
105. **[Med]** What is a dead letter queue (DLQ)? When is it triggered?
106. **[Hard]** How would you handle message idempotency in a consumer?
107. **[Med]** What is consumer lag in Kafka? How do you monitor and fix it?

Observability & Resilience

108. **[Med]** Difference between logging, metrics, and tracing (the 3 pillars of observability).
109. **[Med]** How does distributed tracing work? Explain Zipkin/Jaeger with trace IDs.
110. **[Med]** What is a health check endpoint? How does Spring Actuator /health work?
111. **[Med]** What is retry with exponential backoff? When should you NOT retry?
112. **[Med]** How do you implement bulkhead pattern in microservices?

Docker, Kubernetes & Cloud

113. **[Med]** Difference between Docker container and a VM. What is a Docker image layer?
114. **[Med]** What is a Kubernetes Pod, Deployment, Service, and Ingress?
115. **[Med]** Explain horizontal pod autoscaling (HPA) in Kubernetes.
116. **[Med]** What is a ConfigMap vs Secret in Kubernetes?
117. **[Hard]** How would you do zero-downtime deployment in Kubernetes (rolling update, blue-green, canary)?
118. **[Med]** What is a liveness probe vs readiness probe in Kubernetes?
119. **[Med]** Explain CAP theorem. What does it mean for your distributed system?
120. **[Hard]** How does a service mesh (Istio) work? What problems does it solve?

4. Data Structures & Algorithms

Arrays & Strings

121. **[Easy]** Two Sum — find two indices that add up to target. O(n) with HashMap.
122. **[Med]** Longest substring without repeating characters. (Sliding Window)
123. **[Med]** Product of array except self. Without using division.
124. **[Med]** Container with most water. (Two pointers)
125. **[Med]** Spiral matrix — print a 2D matrix in spiral order.
126. **[Med]** Rotate matrix 90 degrees in-place.
127. **[Easy]** Maximum subarray sum. (Kadane's algorithm)
128. **[Med]** Minimum window substring. (Sliding Window — hard variant)
129. **[Med]** Group anagrams together. (HashMap + sort key)
130. **[Med]** Longest palindromic substring. (Expand around center)

Linked Lists

131. **[Easy]** Reverse a linked list — iterative and recursive.
132. **[Med]** Detect a cycle in linked list. (Floyd's algorithm)
133. **[Med]** Merge two sorted linked lists.
134. **[Hard]** Merge K sorted linked lists. (PriorityQueue / divide and conquer)
135. **[Med]** Find the middle of a linked list in one pass.
136. **[Med]** Remove Nth node from end of list.
137. **[Hard]** Flatten a multilevel doubly linked list.

Trees & Binary Search Trees

138. **[Easy]** Maximum depth / height of a binary tree.
139. **[Med]** Diameter of a binary tree (max leaf-to-leaf path).
140. **[Med]** Lowest common ancestor (LCA) of BST and general binary tree.
141. **[Med]** Level-order traversal (BFS with queue).
142. **[Med]** Validate a binary search tree.
143. **[Hard]** Serialize and deserialize a binary tree.
144. **[Med]** Binary tree right side view.
145. **[Med]** Kth smallest element in a BST.
146. **[Hard]** Maximum path sum in a binary tree.

Graphs

147. **[Med]** Number of islands. (DFS/BFS on a grid)
148. **[Med]** Course schedule — detect cycle in a directed graph. (Topological sort)
149. **[Med]** Clone a graph. (BFS + HashMap)
150. **[Hard]** Word ladder — shortest transformation sequence. (BFS)
151. **[Hard]** Alien dictionary — derive character order. (Topological sort)
152. **[Med]** Find if path exists in a graph. (Union-Find / BFS)

153. **[Hard]** Dijkstra's shortest path algorithm — implement with PriorityQueue.

Dynamic Programming

154. **[Easy]** Climbing stairs — how many ways to reach step n?
155. **[Easy]** House robber — max money without robbing adjacent houses.
156. **[Med]** Coin change — minimum coins to make amount.
157. **[Med]** Longest common subsequence (LCS).
158. **[Hard]** 0-1 Knapsack problem.
159. **[Hard]** Edit distance (Levenshtein distance).
160. **[Med]** Word break — can you segment a string using a dictionary?
161. **[Hard]** Burst balloons — maximum coins. (Interval DP)

Stacks, Queues & Heaps

162. **[Easy]** Valid parentheses — check balanced brackets.
163. **[Med]** Daily temperatures — next warmer day for each day. (Monotonic stack)
164. **[Med]** Implement LRU Cache. (LinkedHashMap or custom DoublyLinkedList + HashMap)
165. **[Med]** Top K frequent elements. (Bucket sort / MinHeap)
166. **[Hard]** Sliding window maximum. (Deque / Monotonic queue)
167. **[Hard]** Median from a data stream. (Two heaps: MaxHeap + MinHeap)
168. **[Med]** Min stack — supports push, pop, getMin in O(1).

Recursion & Backtracking

169. **[Med]** Generate all valid parentheses combinations for N pairs.
170. **[Med]** Subsets / power set of an array.
171. **[Med]** Permutations of an array.
172. **[Med]** Combination sum — find all combos that sum to target.
173. **[Hard]** Sudoku solver. (Backtracking with constraint)
174. **[Hard]** N-Queens problem.
175. **[Med]** Jump Game — can you reach the last index? (Greedy + DP)

5. SQL & Databases

SQL Queries

176. **[Easy]** Write a query to find the second highest salary from an Employee table.
177. **[Med]** Difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN with examples.
178. **[Med]** Write a query to find employees who earn more than their manager.
179. **[Med]** What is a self-join? Write an example query.
180. **[Med]** Difference between WHERE and HAVING clauses. When can you not use WHERE?
181. **[Med]** Write a query to find duplicate records in a table.
182. **[Hard]** Write a query using window functions (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD).
183. **[Med]** What is a CTE (Common Table Expression)? Write a recursive CTE to traverse a hierarchy.
184. **[Easy]** Difference between UNION and UNION ALL.
185. **[Med]** Write a query to pivot rows into columns.
186. **[Hard]** Find the running total of sales by date using window functions.
187. **[Med]** How does GROUP BY work with aggregate functions? What are the restrictions?

Database Design & Performance

188. **[Med]** Explain database normalization — 1NF, 2NF, 3NF, BCNF with examples.
189. **[Med]** What is a database index? How does a B-Tree index work?
190. **[Med]** Clustered vs non-clustered index — difference and use cases.
191. **[Hard]** How would you optimize a slow SQL query? What tools/steps would you use?
192. **[Med]** What is a covering index? How does it eliminate key lookups?
193. **[Med]** What is the N+1 query problem in ORM? How do you fix it in JPA?
194. **[Med]** Explain ACID properties with examples.
195. **[Hard]** Difference between optimistic and pessimistic concurrency control.
196. **[Med]** What is a database transaction isolation level? Explain Read Committed, Repeatable Read, Serializable.
197. **[Med]** What is database sharding? Horizontal vs vertical partitioning.
198. **[Med]** When would you use NoSQL (MongoDB/Redis/Cassandra) over RDBMS?

6. Design Patterns

Creational Patterns

199. **[Easy]** Singleton — thread-safe implementation (double-checked locking, enum).
200. **[Med]** Factory vs Abstract Factory — difference with examples.
201. **[Med]** Builder pattern — when to use over constructor? Show implementation.
202. **[Med]** Prototype pattern — deep copy vs shallow copy.

Structural & Behavioral Patterns

203. **[Med]** Strategy pattern — replace if-else chains. Show a payment example.
204. **[Med]** Observer pattern — explain with a real-world pub/sub example.
205. **[Med]** Decorator pattern — how does Java I/O use it?
206. **[Med]** Proxy pattern — types: virtual, remote, protection proxy.
207. **[Hard]** Chain of Responsibility — middleware pipeline example.
208. **[Med]** Template Method pattern — explain with an example.
209. **[Med]** Adapter pattern vs Facade pattern — what's the difference?
210. **[Med]** Command pattern — how does it enable undo/redo?

7. Low-Level Design (LLD)

Design Problems (common in interviews)

211. **[Hard]** Design a Parking Lot system (classes, inheritance, state machine for slots).
212. **[Hard]** Design an LRU Cache supporting O(1) get and put.
213. **[Hard]** Design an Elevator/Lift system for a building.
214. **[Hard]** Design a Library Management System.
215. **[Hard]** Design a URL Shortener (tinyurl.com) — class design + encoding strategy.
216. **[Hard]** Design a Snake and Ladder game.
217. **[Hard]** Design a Splitwise-like expense sharing system.
218. **[Hard]** Design a Food Delivery system (Zomato/Swiggy) focusing on order management.

8. Target / Rakuten / Wells Fargo — Company-Specific

Target-Specific DSA

219. **[Med]** Spiral matrix traversal — print an M×N matrix in spiral order.
220. **[Med]** Maximum distance between two leaf nodes (diameter) of a binary tree.
221. **[Med]** Longest common substring from a set of strings.
222. **[Med]** Merge K sorted linked lists using a PriorityQueue.
223. **[Med]** Serialize and deserialize a binary tree.
224. **[Med]** Top K frequent elements using a MinHeap.

Rakuten-Specific DSA

225. **[Easy]** String repetition — check if string S = k repetitions of a pattern. Optimize to O(n).
226. **[Med]** Jump Game — greedy approach to determine if last index is reachable.
227. **[Med]** Generate all valid parentheses — backtracking approach.
228. **[Med]** Number of islands — DFS/BFS on a grid.
229. **[Med]** Coin change — minimum number of coins (bottom-up DP).
230. **[Med]** Find duplicate in array using Floyd's cycle detection (no extra space).

Wells Fargo-Specific DSA

231. **[Med]** Minimum number of moves to collect all keys in a grid (BFS + bitmask DP).
232. **[Easy]** Valid parentheses checker using a stack.
233. **[Med]** Merge intervals — merge all overlapping intervals.
234. **[Med]** Lowest common ancestor of a BST — iterative approach.
235. **[Easy]** Climbing stairs — dynamic programming tabulation.
236. **[Med]** Find if a valid path exists between two nodes in an undirected graph.